

[复习主页](#) · [真题与考点](#) · [课程计划](#)

自适应 Huffman 编码

静态 Huffman 得把整篇文章读两遍——先数频率,再编码。可如果数据边到边编、根本不许你先看一眼呢?自适应 Huffman 就是为这种"一遍过"的场景而生。

直觉类比

静态 Huffman 像是考前划重点:你得先把全书看完、统计出哪些词高频,才能定出一套最优代码表。而自适应 Huffman 像是边听课边记笔记:老师每讲一个词,你就把它的出现次数 +1,并随手调整你的速记符号——讲到后面,常出现的词自然拥有了更短的记号。你从来没有"预习",却让代码始终贴合当前的词频。

1. 核心定义

自适应 Huffman 编码 (Adaptive / Dynamic Huffman Coding)

编码器和解码器不预先知道符号频率,而是从一棵空(或初始)编码树出发,每处理一个符号就实时更新这棵 Huffman 树。编码与解码各自独立地维护同一棵树,因此无需传输码表。又称动态 Huffman,代表算法 FGK 与 Vitter。

兄弟性质 (Sibling Property)

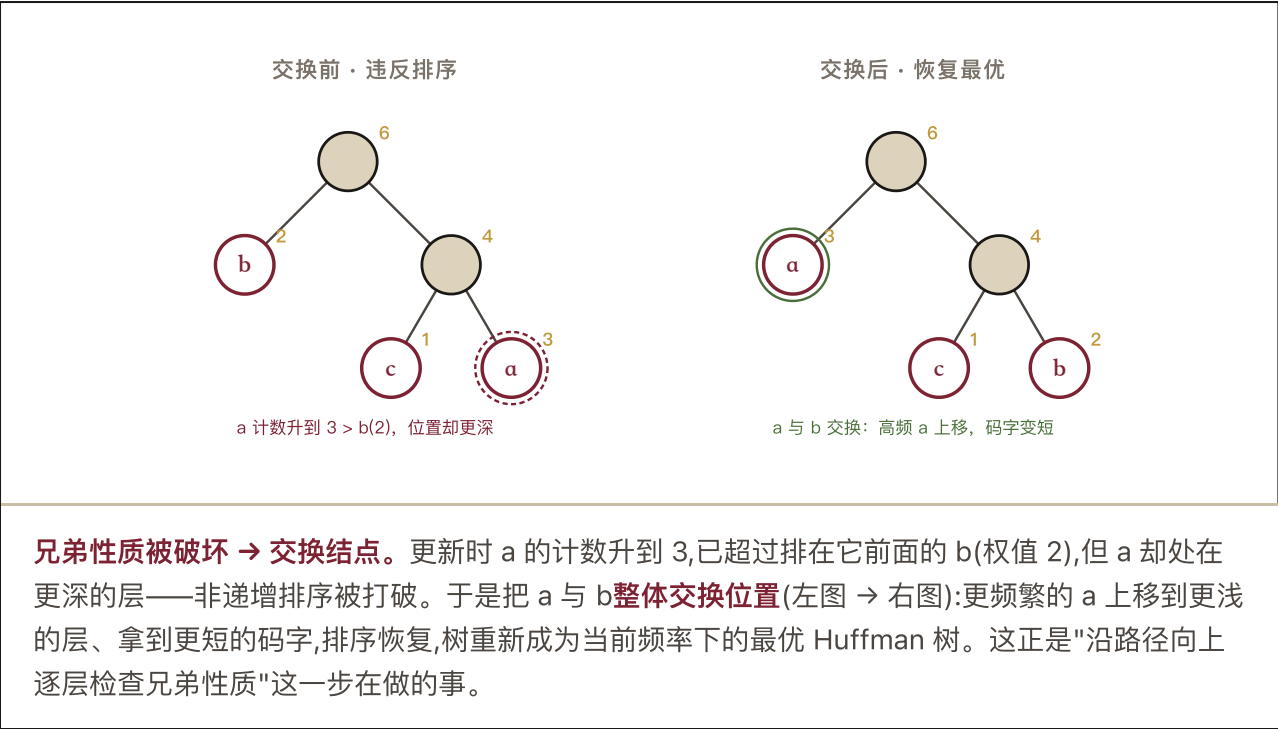
Gallager 给出的判据:一棵二叉编码树是 Huffman 树当且仅当——把所有结点(含内部结点)按权值非递增排成一列时,每个结点都与它的兄弟相邻,且这个序列的权值单调不减。这是"维护最优性"的检验标准:只要它成立,树就仍是最优的。

更新 (Update) 过程

读入一个符号后:①把该叶子的权值 +1;②沿路径向上到根,逐层检查兄弟性质是否被破坏;③若某结点的权值超过了序列中排在它前面的结点,就把它与那个结点交换位置,恢复排序;④继续把父结点权值 +1 并向上重复,直到根。新出现的符号则通过一个特殊的 NYT (Not-Yet-Transmitted,尚未传输)空叶子引入。



NYT 结点如何"分裂"引入新符号。虚线圈是 NYT (尚未传输)空叶子,权值恒为 0。当一个**从未出现过的**符号 x 到来,就把当前 NYT 一分为二:左边保留一个新的 NYT(仍为 0),右边挂上新叶子 x (权值 1),二者共用一个权值为 1 的父结点。读图:金色小数字是权值,酒红实心圈是已知符号叶子。这样这棵树就"长出"了一片新叶,而 NYT 始终留在树上等待下一个新符号。



2. 静态 vs 自适应 Huffman

这是本节的考点核心。两者用的都是"频率越高、码字越短"的同一思想,区别全在频率从哪来、树何时建、码表要不要传:

比较项	静态 HUFFMAN	自适应 HUFFMAN
是否需先验频率	需要——必须先扫全文统计频率	不需要——从零开始边编边学
扫描遍数	两遍(先统计,再编码)	一遍(单遍 / 在线编码)
是否需传码表	需要——解码端要拿到频率表/码表	不需要——收发两端各自实时重建同一棵树
时间复杂度	建树 $O(n \log n)$ (n 为符号种类); 编码 $O(N)$	每处理一个符号 $O(\text{树高}) \approx O(\log n)$,更新有额外开销
适用场景	数据可整体预读、频率稳定	流式/实时数据、频率未知或随时间漂移

静态 Huffman · 两遍



自适应 Huffman · 一遍



两遍 vs 一遍:数据流的对比。上排静态 Huffman 必须先扫一整遍统计频率、建表,才能第二遍编码,最后还要把**码表连同比特流**一起发出去。下排自适应 Huffman 把"编码"和"更新树"揉进同一次遍历——每来一个符号就即时输出码字并调整树(左侧回环箭头表示对每个符号重复),因此只需**一遍**,且只传比特流、**零码表**开销。这张图正是上面表格前三行的可视化。

3. 为什么需要自适应

它并不是为了"压得更狠"——单纯从压缩率看,静态 Huffman 在已知频率时已是最优。自适应的价值全在约束条件下:

- 无需预知频率。网络传输、实时采集的数据是"流",到来之前你根本无从统计。静态 Huffman 在这里直接失效,因为它连第一遍扫描都做不到。
- 单遍编码(**one-pass**)。只扫一遍就能完成压缩,省掉了"统计一遍 + 编码一遍"里的第一遍,对超大文件或延迟敏感的场景是硬需求。
- 省掉码表开销。静态 Huffman 必须把码表(或频率)一并发给解码端,短消息里这张表的开销可能盖过压缩收益。自适应两端各自重建,零码表传输。
- 自动跟随频率漂移。树随数据动态调整,前半段和后半段词频不同也能各自贴合。

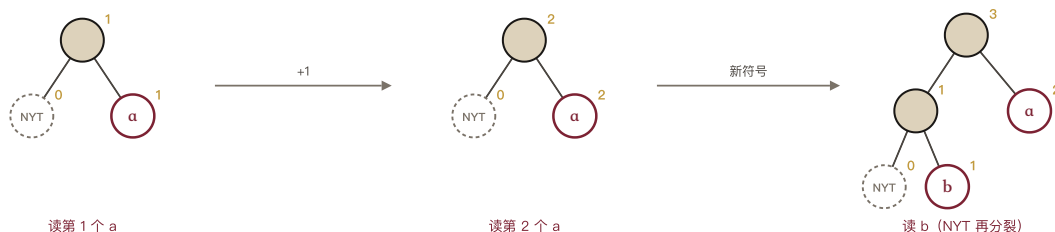
代价是:实现更复杂,且每符号都要维护树、对误码更敏感——一旦某个 bit 出错,两端的树就分道扬镳,后续全乱。

4. 编码—更新流程演示

设初始树只有一个 NYT 空叶子(权值 0)。收发两端都从这里出发,亦步亦趋。为看清"更新"这一动作,我们重点跟踪权值与结点排序。

- 1 读入第 1 个 'a'(首次出现)。树里还没有 'a',于是先输出当前 NYT 的路径码(此刻在根,路径为空),再附上 'a' 的固定编码告诉解码端"这是新符号 a"。然后把 NYT 叶子分裂:左边留一个新的 NYT,右边挂上新叶子 'a'(权值 1),它们的父结点权值 1。
- 2 读入第 2 个 'a'(已存在)。直接输出 'a' 当前的路径码(此时很短)。随后进入更新:'a' 的权值 $1 \rightarrow 2$,再沿路径向上,把父结点权值也 $+1$ 。每一步都检查兄弟性质——本例结点少,排序未被破坏,无需交换。此刻 'a' 的权值已明显高于 NYT,树自动向"a 更短"的方向倾斜。
- 3 读入 'b'(首次出现)。同第 1 步:输出当前 NYT 的路径码 + 'b' 的固定编码。再次分裂 NYT,挂上新叶子 'b'(权值 1)。
- 4 此时若某个新符号的权值增长后超过了排在它前面的结点,就会触发结点交换以恢复非递增排序——这正是"兄弟性质"起作用的地方,保证每一步之后树都仍是当前频率下的最优 Huffman 树。

关键结论:整个过程只读了一遍输入,也没有传任何码表。解码端拿到比特流后,执行与编码端完全相同的分裂与更新动作,便能同步重建出同一棵树,从而正确还原 "a a b"。这就是"边编边学"的全部秘密。



编码 "a a b" 时树的动态生长。三张快照对应上面例题的三步。① 读第 1 个 a: NYT 分裂, 长出 a (权值 1)。② 读第 2 个 a: a 的权值 $1 \rightarrow 2$, 父结点也 $+1$; 本例结点少, 排序未被破坏, **无需交换**。③ 读 b: 再次分裂 NYT, 挂上 b (权值 1), 根权值累计到 3。全程**只读一遍输入**、**不传码表**; 解码端执行完全相同的分裂与 $+1$ 动作, 便能同步重建这三棵一模一样的树。金色数字为权值, 虚线圈为 NYT。

自测题

下列关于自适应 **Huffman** 相对于静态 **Huffman** 优势的说法,哪一项是正确的?

- A. 自适应 Huffman 的压缩率一定严格优于静态 Huffman
- B. 自适应 Huffman 无需预先统计符号频率,可单遍编码且无需传输码表
- C. 自适应 Huffman 不需要在编码器和解码器之间保持任何一致性
- D. 静态 Huffman 只需扫描数据一遍,而自适应需要两遍

▼ 答案与解释

答案:B

B 正确:自适应 Huffman 的三大标志性优势就是——不需先验频率、单遍 (one-pass) 编码、无需传码表(两端各自实时重建)。

A 错:在频率已知时静态 Huffman 已是最优,自适应不保证压得更狠;它的价值在于适应"频率未知/流式"的约束,而非提升压缩率。

C 错:恰恰相反,收发两端必须执行完全相同的更新动作、维护同一棵树,否则无法解码;这也是它对误码敏感的原因。

D 错:说反了——静态需两遍(先统计再编码),自适应只需一遍。

下一步 → Huffman 家族到此为止。进入 [0011·LZW 字典编码](#),换一条思路:不靠频率,而是把重复出现的**字符串**存进字典、用短码代替——GIF 与早期压缩的功臣。